React js
Lecture 4

By Mona Soliman

Agenda

- Recap last lecture points
- React router dom (loaders , errorElement)
- Interceptors
- What is Redux ?
- Redux components
- Getting started with store
- Useful extensions
- Questions!

Error Element

ErrorElement :

In react-router-dom, the ErrorElement is a component used to handle and display errors that occur during route matching or while rendering route components. It is typically used within the context of a Route to catch errors that might be thrown by the components associated with that route.

```
const router = createBrowserRouter([
  {
    path: "/",
    element: <Home />,
  },
  {
    path: "about",
    element: <About />,
    errorElement: <ErrorPage />, // ErrorElement defined here
  },
]);

function App() {
  return <RouterProvider router={router} />;
}

export default App;
```

Loader

In react-router-dom, a loader is a function associated with a route that allows you to fetch data before rendering a component. This function is called when the route is matched, and the data it returns is passed to the route's element via the useLoaderData hook. loaders are useful for ensuring that data is available before a component is rendered, improving the user experience by avoiding intermediate loading states within the component itself.

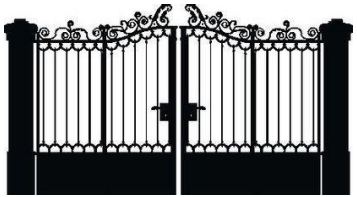
```
const router = createBrowserRouter([
  {
    path: "/user",
    element: <User />,
    loader: fetchUserData, // Loader function defined here
  },
]);
```

Action(Self study)

In react-router-dom, an action is a function associated with a route that handles data mutations, such as form submissions, before navigating to the route's element. This can be used to process form data, perform side effects like saving data to a server, or handle other non-navigation actions. Actions allow you to encapsulate side effects and mutations in a way that integrates seamlessly with the routing system.

https://reactrouter.com/en/main/route/action

Interceptor

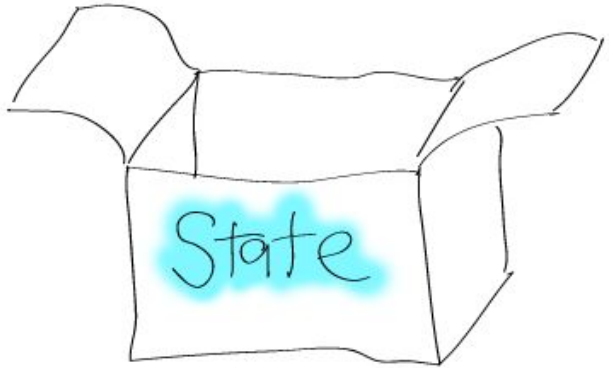


Axios interceptors are the default configurations that are added automatically to every request or response that a user receives. It is useful to check response status code for every response that is being received.

```
axios.interceptors.request.use
(
    (req) => {
       // Add configurations
here
       return req;
    },
    (err) => {
       return
Promise.reject(err);
    }
);
```

```
axios.interceptors.response.use(
    (res) => {
       // Add configurations here
       if (res.status === 201) {
          console.log('Posted
Successfully');
       }
       return res;
    },
    (err) => {
       return Promise.reject(err);
    }
);
```

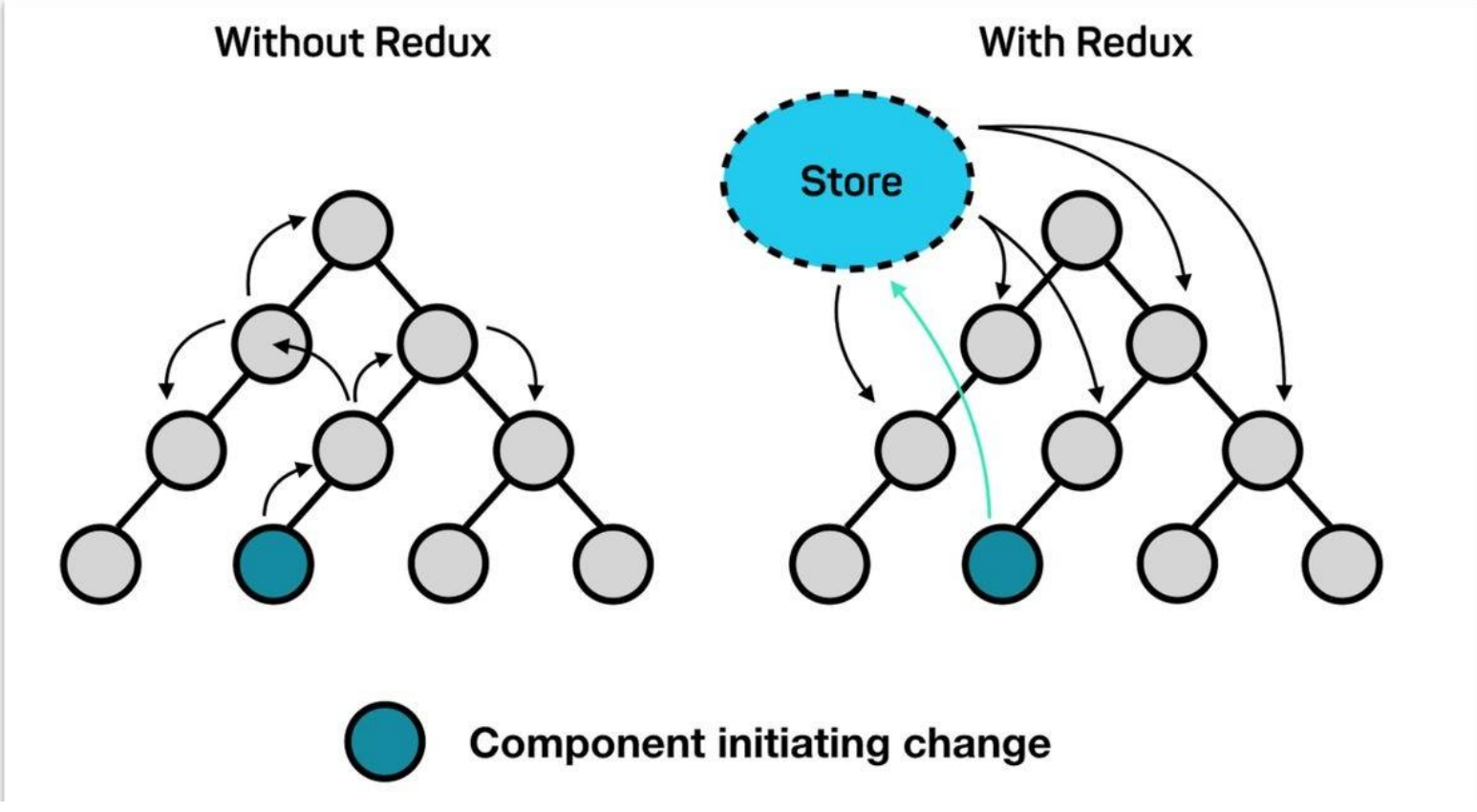
Types of State



Local State:
Local state refers to state that is managed within a specific component. It is only accessible and relevant to that component. Local state is used for handling UI-related data, such as form inputs, toggles, or any other data that does not need to be shared across multiple components.

Global State:
Global state refers to state that is shared across multiple components or throughout the entire application. Global state is necessary when multiple components need to access or update the same piece of data. This type of state is typically managed using external libraries like Redux , Zustand, or the context api in React.

Redux


Without Redux
With Redux
Store
Component initiating change

Props Drilling

Props drilling is a term used in React to describe the process of passing data from a parent component to deeply nested child components through multiple layers of intermediate components. This happens when a piece of data or a function needs to be accessed by a component that is several levels deep in the component tree, but each intermediate component has to pass the data or function down via props.

Redux

Redux is a predictable state container for JavaScript apps.You can use Redux together with React , or with any other view library.the state of your application is kept in store. And any component can access the state from the store.

to use redux Toolkit in your application :

npm install @reduxjs/toolkit react-redux

Redux Toolkit

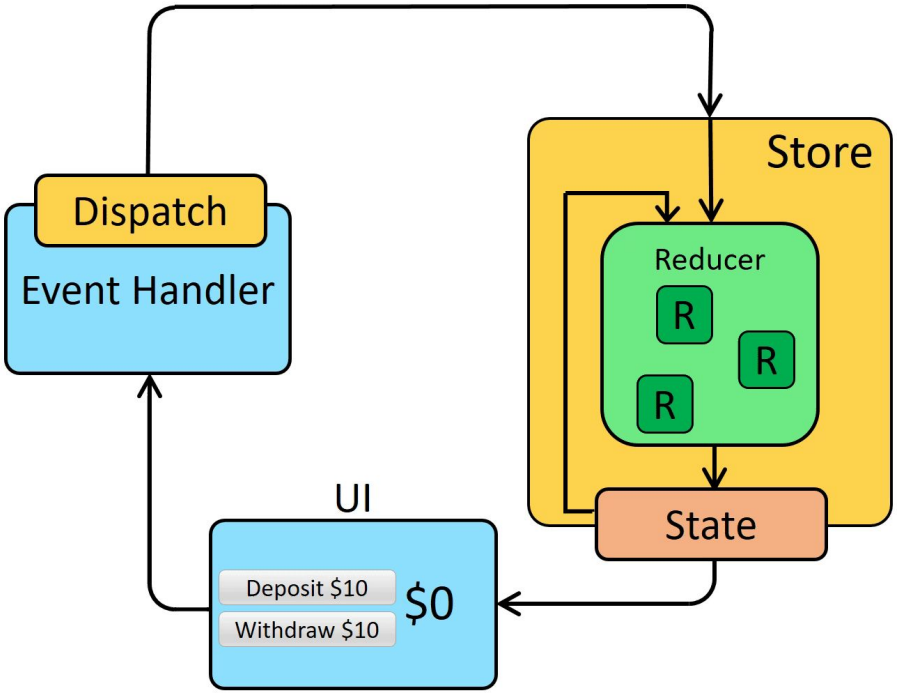
The Redux Toolkit package is intended to be the standard way to write Redux logic. It was originally created to help address three common concerns about Redux:

- "Configuring a Redux store is too complicated"
- "I have to add a lot of packages to get Redux to do anything useful"
- "Redux requires too much boilerplate code"

Redux

There are three building parts

- Actions
- Reducers
- Store



Lab

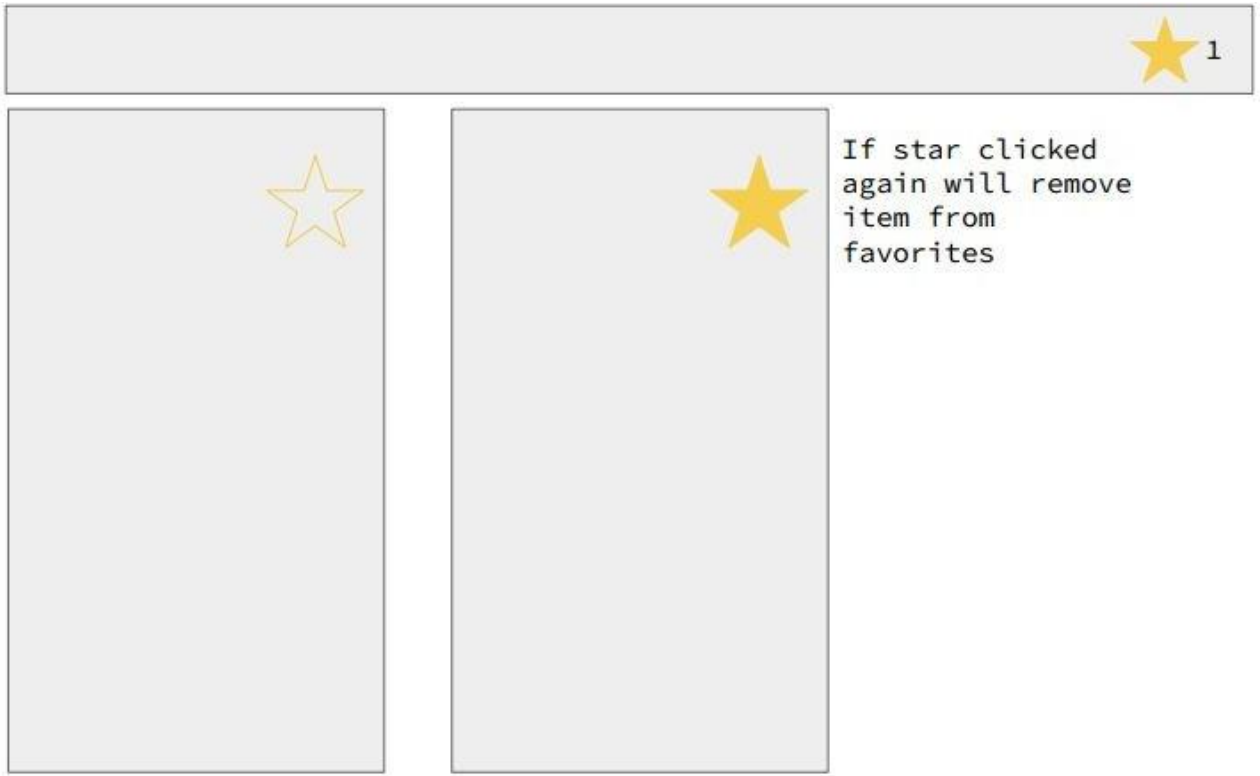
Lab

Create redux cycle to add movies to favorites:

- if movie added to favorites star or heart icon should be styled with filled color.

- If movie not in the favorites icon should be bordered.
- User can go to favorites page
- Favorites count should appear in navbar
- User can remove movies from favorites page

LAB



LAB

Thank you